

Enterprise Programming (Course Slides)

Module4

Refer for detailed notes :-

<https://github.com/sagaruppuluri/EP/blob/main/Module4/Readme.md>

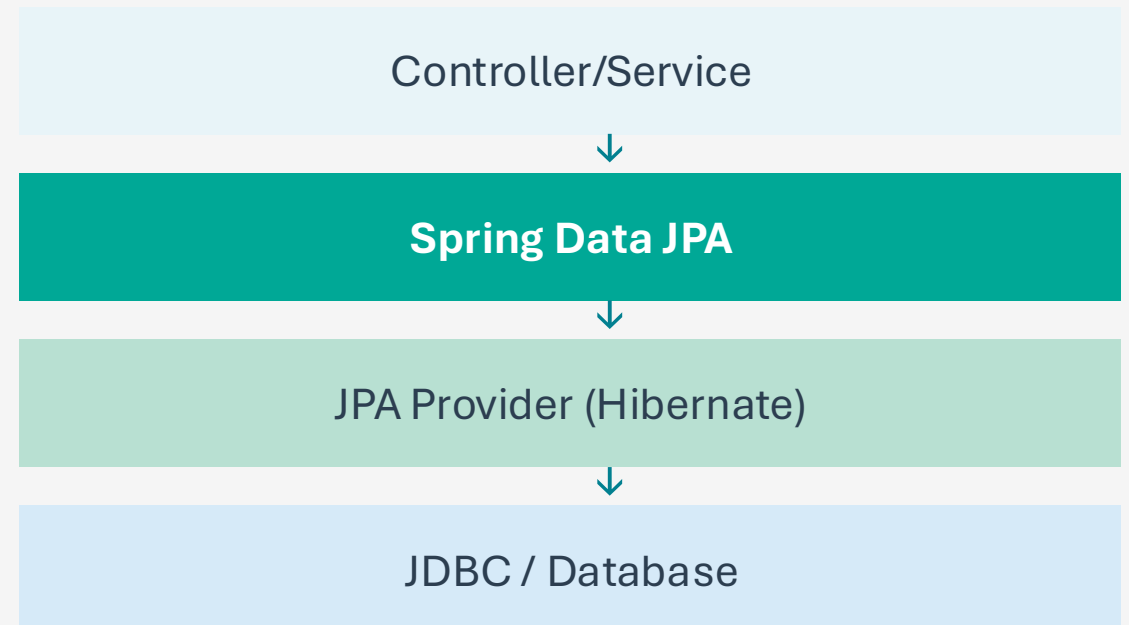
What is Spring Data JPA?

Spring Data JPA is a powerful abstraction layer over JPA (Java Persistence API) that eliminates boilerplate code and provides automatic repository implementations for common database operations.

Key Benefits

- ✓ Reduces 90% of data access code
- ✓ Type-safe query methods
- ✓ Automatic CRUD implementations
- ✓ Built-in pagination & sorting

Architecture



1. Entity: *JPA entities are POJOs representing database tables*

```
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Column(precision = 10, scale = 2)
    private BigDecimal price;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "category_id")
    private Category category;

    @CreatedDate
    private LocalDateTime createdAt;

    // Getters and Setters
}
```

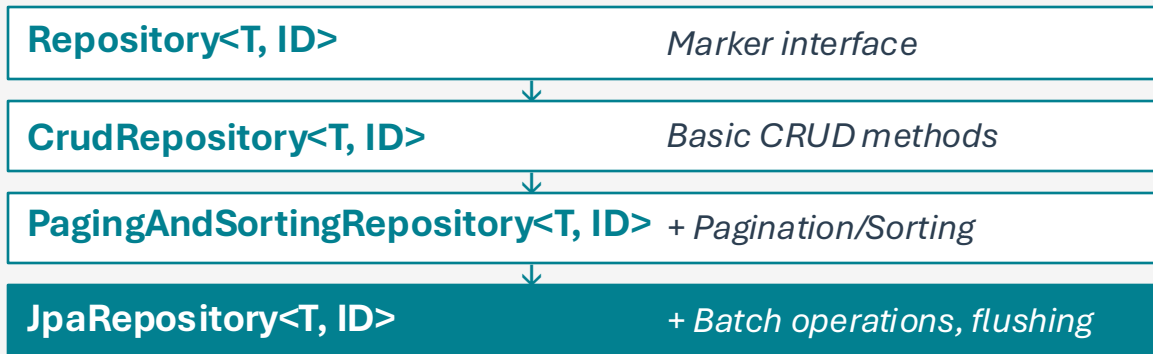
2. Repository Interface

Extend JpaRepository for automatic CRUD operations

```
@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

    // Automatically provides:
    // - save(), findById(), findAll(), deleteById()
    // - Plus custom method declarations (shown in next slides)
}
```

Repository Hierarchy



Key Features

- No implementation needed
- Query method derivation
- Custom queries with @Query
- Transaction management

3. CRUD Operations

Built-in methods from JpaRepository

Operation	Method	Return Type
Create	<code>save(entity)</code>	Product
Read	<code>findById(id)</code>	Optional<Product>
Read All	<code>findAll()</code>	List<Product>
Update	<code>save(entity)</code>	Product
Delete	<code>deleteById(id)</code>	void
Count	<code>count()</code>	long
Exists	<code>existsById(id)</code>	boolean

Example Usage

```
Product product = new Product("Laptop", new BigDecimal("999.99"));
productRepository.save(product); // Create
Optional<Product> found = productRepository.findById(1L); // Read
```

4. Custom Query Methods

Automatic query generation from method names

```
public interface ProductRepository extends JpaRepository<Product, Long> {  
  
    List<Product> findByName(String name);  
  
    List<Product> findByNameContainingAndPriceLessThan(String name, BigDecimal price);  
  
    Optional<Product> findFirstByOrderByCreatedDateDesc();  
}
```

Common Query Keywords

findBy	Select query
countBy	Count results
deleteBy	Delete query
And / Or	Logical operators
LessThan / GreaterThan	Comparisons
Containing / Like	Pattern matching
OrderBy...Asc/Dsc	Sorting

Advantages

- ✓ Type-safe at compile time
- ✓ No SQL required
- ✓ Self-documenting code
- ✓ Automatic implementation

5. @Query Annotation (JPQL)

Write custom JPQL queries for complex operations

```
@Query("SELECT p FROM Product p WHERE p.price BETWEEN :min AND :max")
List<Product> findByPriceRange(@Param("min") BigDecimal min,
                              @Param("max") BigDecimal max);

@Query("SELECT p FROM Product p JOIN FETCH p.category WHERE p.active = true")
List<Product> findActiveProductsWithCategory();

@Query("SELECT new com.example.dto.ProductDTO(p.id, p.name, p.price) " +
        "FROM Product p WHERE p.category.id = :categoryId")
List<ProductDTO> findProductDTOsByCategory(@Param("categoryId") Long id);

@Modifying
@Query("UPDATE Product p SET p.active = false WHERE p.createdDate < :date")
int deactivateOldProducts(@Param("date") LocalDateTime date);
```

JPQL Characteristics

- Operates on entity objects, not tables
- Database-independent syntax
- Supports joins, projections, subqueries
- Use @Param for named parameters

Important

- Use @Modifying for UPDATE/DELETE
- @Transactional required for modifying
- Entity names are case-sensitive

6. Native SQL Queries

Use database-specific SQL when needed

```
@Query(value = "SELECT * FROM products WHERE category_id = ?1 " +  
            "AND stock > 0 ORDER BY created_date DESC LIMIT ?2",  
        nativeQuery = true)  
List<Product> findTopAvailableByCategory(Long categoryId, int limit);  
  
@Query(value = "SELECT p.*, c.name as category_name " +  
            "FROM products p INNER JOIN categories c ON p.category_id = c.id " +  
            "WHERE p.price > :minPrice",  
        nativeQuery = true)  
List<Object[]> findProductsWithCategoryNative(@Param("minPrice") BigDecimal price);  
  
@Modifying  
@Query(value = "DELETE FROM products WHERE created_date < :cutoffDate",  
        nativeQuery = true)  
int deleteOldProductsNative(@Param("cutoffDate") LocalDateTime date);
```


When to Use Native Queries?

Scenario	JPQL	Native SQL
Database-specific functions	✗	✓
Complex joins with CTEs	✗	✓
Performance optimization	⚠	✓
Database portability	✓	✗
Type safety	✓	⚠

7. Transaction Management

Ensuring data consistency with @Transactional

```
@Service
@Transactional(readOnly = true)
public class ProductService {

    private final ProductRepository repository;

    // Read-only by default
    public Product findById(Long id) {
        return repository.findById(id)
            .orElseThrow();
    }

    // Override for write operations
    @Transactional
    public Product create(Product product) {
        validateProduct(product);
        return repository.save(product);
    }

    @Transactional
    public void deleteById(Long id) {
        repository.deleteById(id);
    }
}
```

Attributes

- readOnly = true/false
- timeout = seconds
- isolation = LEVEL
- propagation = TYPE
- rollbackFor = Exception.class

Propagation

REQUIRED (default)
REQUIRES_NEW
NESTED
SUPPORTS

ACID Properties

A

Atomicity

All or nothing

C

Consistency

Data integrity maintained

I

Isolation

Concurrent transactions
don't interfere

D

Durability

Changes persist after commit

8. Pagination & Sorting

Efficiently handle large datasets

```
public interface ProductRepository extends JpaRepository<Product, Long> {  
    Page<Product> findByCategory(String category, Pageable pageable);  
}
```

Service Layer Usage

```
@Service  
public class ProductService {  
  
    public Page<Product> getProducts(  
        int page, int size, String sortBy) {  
  
        Pageable pageable = PageRequest.of(  
            page, size,  
            Sort.by(sortBy).descending()  
        );  
  
        return repository.findAll(pageable);  
    }  
}
```

Page<T> Methods

- getContent() - List<T>
- getTotalElements() - long
- getTotalPages() - int
- getNumber() - int
- getSize() - int
- hasNext() - boolean
- hasPrevious() - boolean
- isFirst() / isLast()

Controller Example

```
@GetMapping("/products")  
public ResponseEntity<Page<Product>> getProducts(@RequestParam int page, @RequestParam int size) {  
    return ResponseEntity.ok(productService.getProducts(page, size, "createdAt"));  
}
```